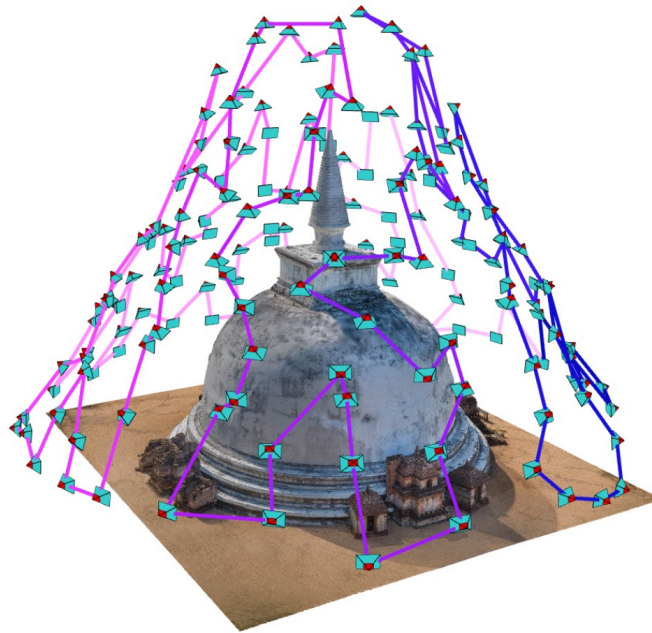


1. Introduction

Unmanned Aerial Vehicles (UAVs or drones) are increasingly being used for surveying terrestrial structures, including those at sites of cultural significance. The survey is intended to acquire enough high-quality images to enable photogrammetric reconstruction of a detailed 3D point/mesh model of the structure. Flight path planning is a critical part of this, and is often done manually with a human guiding the UAV as it acquires images. There has been extensive work to automate such planning, utilising complex computer vision algorithms that can accept a simplified 3D model and produce an "optimal" flight path—which avoids excessive flight times and does not require a lot of images.



This project involves creating a 3D visualisation system for UAV flight trajectories produced by a UAV flight path simulator. The system should allow users to preview flight paths, compare different trajectories (typically produced by different algorithms), and export mission data for real-world deployment.

You will need to include all implementation steps, design rationale, challenges encountered, and solutions developed to be documented comprehensively in your project report. Appropriate usage instructions should also be included in your project GitLab README).

Core Requirements

The basic suggested approach (which you are free to modify, although you should be able to explain your approach clearly, and you must credit any external sources used) is as follows:

- 1) Design a system architecture that organizes the 3D rendering, user interaction, and data processing components. Define your class hierarchy ensuring modules can be

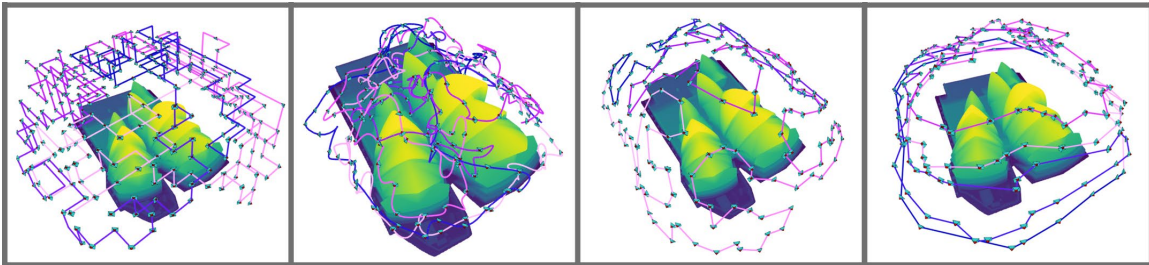
modified independently with clear interfaces between them. Document your architectural decisions and explain how your design addresses the system's requirements.

2) Implement 3D scene model loading with texture support. Your system should handle at least one common 3D format (such as OBJ or PLY). You may use libraries like PyRender, Open3D, or Trimesh. Note that different libraries have different strengths.

3) Create a trajectory parsing module to process waypoint data files. A flight path consists of a sequence of waypoints which are distinct positions and orientations of the UAV camera systems along the flight path. Waypoints should be provided in JSON format with position coordinates (x, y, z) and orientation information (yaw, pitch) in degrees. The JSON structure should include a cameras array containing waypoint data, with a waypoint order array to specify the flight sequence (see resource).

4) Initialise the 3D rendering environment with a scene containing the loaded 3D structure (the building/monument being surveyed), ground reference plane, and coordinate axes for spatial orientation. Implement basic camera controls supporting X, Y, Z translation and yaw/pitch rotation.

5) Render the flight trajectory curve using spline interpolation to connect discrete waypoints. This creates a smooth path representation in the 3D scene. Include directional indicators to show flight heading.



6) Visualise waypoint markers at each trajectory point. The markers should reflect camera orientation (yaw and pitch angles). Implement selection highlighting to improve user interaction.

7) Implement first-person view (FPV) preview functionality. When a waypoint is selected, position the camera at that location with the correct orientation. Display the rendered view to simulate the UAV's perspective. This preview should be shown separately from the main 3D view.



- 8) Create a user interface containing the 3D viewport for scene visualization, control panel for camera manipulation and trajectory selection, and a waypoint list showing position and orientation data for each point in the trajectory. All UI operations should remain responsive. Rendering should not block user interaction.
- 9) Develop multi-trajectory comparison functionality to analyze differences between different flight paths. Calculate and display comparative metrics including total path length, total vertical distance (sum of all altitude changes between waypoints), and estimated flight duration. For flight duration estimation, assume a constant UAV cruising speed (e.g., 5 m/s) and include appropriate hover time (e.g., 2-3 seconds) at each waypoint. Present results in a side-by-side comparison interface to help users evaluate which flight path is more efficient for their survey needs.
- 10) Implement interpolation method comparison for individual trajectories. For each flight path, compare linear waypoint connections versus spline interpolation methods. Display both connection methods simultaneously in the 3D view with different visual styles (e.g., different colors or line types). Calculate and display simple comparison metrics: total path length for each method and the number of sharp corners (waypoints with turning angles > 90 degrees) in the linear path versus the smooth curves in the spline path.
- 11) Implement a multi-threaded architecture to prevent UI blocking during rendering operations. Use appropriate thread communication mechanisms. Long operations should display progress feedback. Ensure thread safety.
- 12) Record flight simulations as video files. Support customisable resolution and framerate. Overlay trajectory information and position indicators.

Completing the core work of the project will give you up to 85%.

3. Mastery Work

The remaining 15% of the marks are for work beyond the core requirements. The number of marks awarded will depend on the complexity of the added features. For example:

- Interactive waypoint editing with real-time FPV preview: Allow keyboard adjustment of position (x, y, z) and orientation (yaw, pitch). Add/remove waypoints. Update trajectory in real-time. Implement undo/redo functionality. During interactive editing, provide real-time first-person view updates that reflect the current waypoint's camera perspective as the user modifies position and orientation parameters. The FPV window should update smoothly as adjustments are made.
- Waypoint marker batch optimisation: Implement object pooling to pre-create marker instances. Use parallel updates for transformation matrices. This becomes important for trajectories with hundreds of waypoints.
- Advanced 3D visualisation with pythreejs: Replace or augment the basic visualisation with pythreejs for more sophisticated rendering capabilities. Implement custom shaders for waypoint markers, advanced lighting effects, and real-time shadows. Create interactive widgets for trajectory manipulation directly in the 3D space. Note that while pythreejs offers greater control over rendering and UI customisation, it requires lower-level implementation and more complex state management.
- Advanced user interface design with flexible layout: Implement a customizable UI system that allows users to resize and reposition key interface panels (3D viewport, control panel, waypoint list, FPV preview). Support saving and loading different layout configurations for various workflows. Include basic layout presets such as editing mode, comparison mode, and presentation mode with appropriate panel arrangements and sizes.

4. Code Structure

- Modularise your code to work as a pipeline for each processing stage.
- Components should be replaceable without affecting other modules.
- Use appropriate data structures - NumPy arrays for numerical data, dictionaries for metadata. Consider implementing a plugin architecture for extensibility.

5. Resources

High-quality 3D textured models, sample trajectory files from various planners, and example 3D render setup code will be provided. See Resources folder.